

PROGETTO TECNOLOGIE INTERNET

e-Commerce di Vinili Musicali

Introduzione

Il progetto consiste nello sviluppo di un sito web di tipo e-Commerce, dedicato alla vendita di vinili musicali. L'obiettivo principale è quello di realizzare un'applicazione web completa, composta da frontend, backend e database.

Il sistema simula il funzionamento di un vero negozio online, permettendo agli utenti di consultare un catalogo di prodotti, effettuare ricerche e filtri, gestire un carrello, registrarsi al sito ed effettuare ordini. Sono inoltre previste funzionalità di amministrazione riservate agli utenti con ruolo di admin, che consentono la gestione del catalogo dei prodotti.

Lo scopo del progetto non è quello di realizzare un sistema di pagamento reale, ma di progettare e implementare un'applicazione web funzionante, coerente e sicura, in grado di gestire correttamente i dati e le interazioni tra utente e sistema.

Analisi dei requisiti

Per i **requisiti funzionali** il sistema deve consentire a un utente non autenticato di visualizzare il catalogo vinili e di consultare dettagli di ciascun prodotto. Un utente autenticato deve poter aggiungere prodotti al carrello, modificarne la quantità, procedere al checkout ed effettuare un ordine.

Sono inoltre previste funzionalità specifiche per gli utenti amministratori, che includono la creazione, modifica ed eliminazione dei prodotti presenti nel catalogo.

Per quanto riguarda i **requisiti non funzionali**, il sistema garantisce un livello minimo di sicurezza tramite autenticazione basata su *token JWT*, una buona usabilità grazie a un'interfaccia chiara e reattiva, e prestazioni adeguate a un numero limitato di utenti e prodotti.

Architettura del sistema

L'applicazione segue un'architettura Client-Server, che consente di separare in modo chiaro la logica di presentazione dell'applicazione dalla gestione dei dati, in particolare:

- il **frontend**, sviluppato in *React*, si occupa esclusivamente della presentazione dei dati e dell'interazione con l'utente
- il **backend**, realizzato con *Node.js* ed *Express*, gestisce la logica applicativa e fornisce API REST per l'accesso ai dati
- il **database MongoDB** viene utilizzato per la persistenza dei dati relativi a utenti, prodotti e ordini

La comunicazione tra frontend e backend avviene tramite richieste http e risposte in formato JSON, permettendo un'interazione asincrona e dinamica tipica delle applicazioni web moderne.

BACKEND

Ruolo del backend nell'applicazione

Il backend rappresenta il cuore logico dell'applicazione, occupandosi della gestione dei dati, dell'autenticazione degli utenti e dell'esecuzione delle operazioni principali del sistema. Esso fornisce al frontend un insieme di **API (Application Programming Interface)**, che permettono di accedere in modo controllato alle risorse dell'applicazione (utenti, vinili, carrello e ordini).

Il backend è progettato per essere **stateless**, ovvero che ogni richiesta http viene elaborata in modo indipendente e senza la necessità di memorizzare informazioni sulle interazioni precedenti tra le richieste di un client, in modo da facilitare la scalabilità e migliorare l'affidabilità del server.

Server Express e gestione delle richieste

Il backend è stato sviluppato utilizzando **Node.js** ed **Express**, creando un server http (*src/server.js*) che si occupa di:

- mettersi in ascolto sulla porta dedicata (default 3001)
- intercettare le richieste http provenienti dal frontend
- applicare middleware di autenticazione e autorizzazione
- inoltrare le richieste alle rotte corrette in base all'endpoint richiesto
- restituire risposte in formato JSON

```
//rotta per l'autenticazione
app.use('/api/auth', authRoutes);
//rotta per i vinili
app.use('/api/records', recordRoutes);
//rotta per il carrello
app.use('/api/cart', cartRoutes);
//rotta per gli ordini
app.use('/api/orders', orderRoutes);

//porta di ascolto default 3001
const PORT = process.env.PORT || 3001;
```

Figura 1: definizione delle rotte principali del backend tramite Express

Questa struttura consente di mantenere una chiara separazione logica di routing, i controlli di sicurezza e la logica applicativa, rendendo il backend modulare ed estendibile.

Modellazione dei dati con MongoDB e Mongoose

Il database utilizza **MongoDB** (*config/db.js*), un database NoSQL basato su documenti, scelto per la sua flessibilità nella gestione di dati eterogenei e per la naturale integrazione con applicazioni sviluppate in ambiente JavaScript.

La connessione al database è configurata nel file *config/db.js*. Le principali collection utilizzate dal sistema sono *User*, *Record* e *Order*.

La definizione dei modelli avviene tramite **Mongoose**, una libreria **Object Data Modeling (ODM)** che consente di definire schemi, validare i dati ed interagire con il database in modo strutturato. L'utilizzo di Mongoose permette di mantenere coerenza tra la struttura dei dati e la logica applicativa del backend.

```
const mongoose = require('mongoose');
const bcrypt = require('bcrypt'); // importa bcrypt

// definizione dei campi principali per un utente
const UserSchema = new mongoose.Schema({
  name: { type: String, required: true, unique: false },
  surname: { type: String, required: true, unique: false },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true },
  address: { type: String, required: true },
  city: { type: String, required: true },
  role: { type: String, enum: ['customer', 'admin'], default: 'customer' },
  cart: [
    {
      record: { type: mongoose.Schema.Types.ObjectId, ref: 'Record' },
      quantity: { type: Number, default: 1 }
    }
  ]
});

module.exports = mongoose.model('User', UserSchema);
```

Figura 2: modellazione del carrello dentro la struttura dell'utente

Il carrello è stato modellato come array all'interno del documento *User* (*models/User.js*) in quanto rappresenta uno stato temporaneo e strettamente associato all'utente autenticato. Questa scelta semplifica la struttura dei dati, evitando la creazione di una collection separata e riducendo il numero di query necessarie. In questo modo è possibile recuperare in un'unica operazione sia i dati dell'utente che il contenuto del carrello.

```
//definizione dei campi principali per un ordine
const OrderSchema = new mongoose.Schema({
  user: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  items: [
    {
      record: { type: mongoose.Schema.Types.ObjectId, ref: 'Record', required: true },
      title: { type: String, required: true },
      quantity: { type: Number, required: true },
      price: { type: Number, required: true }
    }
  ]
});
```

Figura 3: l'ordine fa riferimento all'utente e ai prodotti acquistati

La collection *Order* (*models/Order.js*) utilizza riferimenti (*ObjectId*) verso *User* e *Record* (*models/Record.js*) per mantenere la relazione tra ordine, utente e prodotti acquistati. Questo approccio evita duplicazioni inutili di dati e consente di recuperare informazioni correlate tramite l'uso della funzione *populate*. All'interno di ogni ordine, vengono salvati anche il titolo e il prezzo del vinile al momento dell'acquisto, in modo da mantenere la coerenza storica dell'ordine, evitando che eventuali modifiche future ai prodotti influiscano sugli ordini già effettuati.

API REST e logica applicativa

Il backend espone **API REST (Representational State Transfer)** che rappresentano il punto di comunicazione tra frontend e database. Attraverso queste API è possibile gestire tutte le funzionalità principali dell'applicazione, tra cui l'autenticazione degli utenti, la consultazione del catalogo vinili, la gestione del carrello e la creazione degli ordini.

Ogni API riceve una richiesta http, esegue la logica necessaria e restituisce una risposta in formato JSON, garantendo un'interazione standardizzata e indipendente dalla tecnologia utilizzata dal frontend.

Autenticazione e sicurezza (JWT)

L'autenticazione è basata su **JSON Web Token (JWT)**, una soluzione stateless che consente di identificare in modo sicuro gli utenti nelle richieste del backend.

Durante la fase di registrazione, inoltre, la password inserita dall'utente viene cifrata tramite la funzione `bcrypt.hash`, in modo da non memorizzare mai password in chiaro nel database. Questa scelta aumenta il livello di sicurezza del sistema, impedendo sia al server sia agli amministratori di accedere direttamente a informazioni sensibili.

```
router.post('/register', async (req, res) => {
  try {
    const { name, surname, email, password, address, city } = req.body;

    // controllo campi obbligatori
    if (!name || !surname || !email || !password || !address || !city) {
      return res.status(400).json({ message: "Tutti i campi sono obbligatori" });
    }

    // controllo se email già registrata
    const existingUser = await User.findOne({ email });
    if (existingUser) {
      return res.status(400).json({ message: "Email già registrata" });
    }

    // hash password
    const hashedPassword = await bcrypt.hash(password, 10);

    // creazione nuovo utente
    const newUser = new User({
      name,
      surname,
      email,
      password: hashedPassword,
      address,
      city,
      role: 'customer'
    });

    await newUser.save();

    // token JWT
    const token = jwt.sign(
      { userId: newUser._id, role: newUser.role },
      JWT_SECRET,
      { expiresIn: '7d' }
    );

    res.status(201).json({
      token,
      user: {
        id: newUser._id,
        name: newUser.name,
        surname: newUser.surname,
        email: newUser.email,
        role: newUser.role
      }
    });
  } catch (err) {
    console.error("ERRORE REGISTER:", err);
    res.status(500).json({
      message: "Errore nella registrazione",
      error: err.message
    });
  }
});
```

Figura 4: registrazione con hash della password e generazione del token

Quando il frontend chiama un'API protetta (come `/cart` o `/orders`) il token JWT viene incluso nell'header http della richiesta. Prima che questa venga inoltrata alla rotta specifica, entra in gioco il **middleware** "authRequired" (`middleware/authmiddleware.js`), che si occupa di verificare la validità del token.

La verifica avviene tramite la funzione `jwt.verify` e la chiave segreta `JWT_SECRET`, che consente di controllare l'integrità del token e la sua validità temporale. In caso di token mancante, non valido o scaduto, la richiesta viene immediatamente rifiutata con un errore di autorizzazione.

```
// middleware per verificare token JWT
function authRequired(req, res, next) {
  const authHeader = req.headers['authorization'];
  const token = authHeader && authHeader.split(' ')[1];
  if (!token) {
    return res.status(401).json({ message: 'Token mancante' });
  }

  try {
    const decoded = jwt.verify(token, JWT_SECRET);
    // normalizzo in un formato semplice
    req.user = {
      id: decoded.userId, // userId preso dal token
      role: decoded.role // ruolo (customer/admin)
    };

    next();
  } catch (err) {
    return res.status(401).json({ message: 'Token non valido' });
  }
}
```

Figura 5: controllo del token durante il processo di autenticazione

Dopo la verifica positiva del token, le informazioni principali dell'utente vengono inserite nell'oggetto "req.user". Questo permette alle rotte successive di accedere direttamente all'identità e al ruolo dell'utente autenticato, senza dover decodificare nuovamente il token.

La separazione tra **autenticazione** (`authRequired`) e **autorizzazione** (`adminRequired`) rende il sistema più modulare e mantenibile, permettendo di applicare in modo flessibile i controlli di sicurezza alle diverse API in base ai privilegi richiesti.

Gestione del carrello

Il carrello (*routes/cart.js*) è associato a ogni utente ed è memorizzato nel database come **array di oggetti** all'interno del documento *User*. Ogni elemento contiene il riferimento al vinile selezionato e la relativa quantità. Le API dedicate al carrello consentono di:

- Aggiungere prodotti (POST)
- Visualizzare il carrello (GET)
- Modificare le quantità (PUT)
- Rimuovere articoli (DELETE)

Quando un utente aggiunge un vinile al carrello, il backend verifica l'esistenza del prodotto nel database e controlla se il vinile è già presente nel carrello. In tal caso, la quantità viene incrementata; altrimenti, viene inserito un nuovo elemento.

```
// cerco se il record esiste già nel carrello
const existingItem = user.cart.find(
  (item) => item.record.toString() === recordId
);

if (existingItem) {
  existingItem.quantity += qty;
} else {
  user.cart.push({ record: recordId, quantity: qty });
}

await user.save();

// ricarico l'utente con populate per restituire i dettagli completi
const updatedUser = await User.findById(req.user.id).populate('cart.record');
```

Figura 6: gestione del carrello

Dopo aver salvato le modifiche nel documento *User*, il backend ricarica l'utente utilizzando la funzione *populate* sul campo "cart.record". Questo consente di restituire al frontend i dettagli completi del prodotto (come titolo, prezzo e immagine) in un'unica risposta, senza richiedere chiamate aggiuntive. In questo modo, la risposta della rotta */cart/add* è immediatamente utilizzabile per l'aggiornamento dell'interfaccia utente.

Creazione e gestione degli ordini

Le API dedicate alla creazione e gestione degli ordini (*routes/order.js*) permettono:

- La creazione dell'ordine (POST)
- La visualizzazione ordini precedenti (GET)
- La visualizzazione e modifica dello stato degli ordini (solo admin)

```
//creo l'ordine dagli articoli nel carrello
const items = user.cart.map(item => ({
  record: item.record._id,
  title: item.record.title,
  price: item.record.price,
  quantity: item.quantity
}));

//calcolo il totale
const total = items.reduce((sum, item) => sum + item.price * item.quantity, 0);
//creo e salvo l'ordine
const order = new Order({
  user: user._id,
  items,
  total,
  status: 'in elaborazione',
});

await order.save();
//svuoto il carrello dell'utente dopo aver creato l'ordine
user.cart = [];
await user.save();
return res.status(201).json({message: 'Ordine creato con successo', order});
```

Figura 7: gestione degli ordini

Durante la fase di checkout, il backend utilizza il contenuto del carrello dell'utente per creare un nuovo documento *Order* (*models/Order.js*), che include i prodotti acquistati, le quantità e il prezzo al momento dell'acquisto.

Una volta creato l'ordine, il carrello dell'utente viene svuotato, garantendo la corretta separazione tra lo stato temporaneo del carrello e lo stato persistente degli ordini. Anche in questo caso, l'utilizzo dei riferimenti (*ObjectId*) e della funzione *populate* permette di gestire in modo efficiente le relazioni tra utenti, ordini e prodotti, fornendo al frontend dati completi e pronti per la visualizzazione.

Gestione del catalogo vinili (Records) e filtri

Il modulo *routes/records.js* implementa tutte le API REST necessarie alla gestione del catalogo dei vinili nello shop. Questo modulo rappresenta il punto di accesso centrale per la visualizzazione dei prodotti nel frontend, sia nella pagina principale (Home) che nella pagina di dettaglio del singolo vinile (*RecordDetail*).

Le API definite in questo file permettono sia la **consultazione pubblica del catalogo**, con supporto a funzionalità di ricerca e filtraggio, sia le **operazioni di amministrazione** (creazione, modifica ed eliminazione dei vinili), che sono protette tramite autenticazione e controllo dei privilegi.

```
router.get('/', async (req, res) => {
  try {
    //estraggo i parametri di filtro dalla query string
    const {search, genre, minPrice, maxPrice, yearFrom, yearTo} = req.query;
    const filter = {};
    //in base ai parametri, costruisco il filtro (ricerca testuale)
    if (search) {
      filter.$or = [
        { title: { $regex: search, $options: 'i' } },
        { artist: { $regex: search, $options: 'i' } }
      ];
    }
    //applico altri filtri nella Navbar
    if (genre) { filter.genre = genre; }
    if (minPrice) { filter.price = { ...filter.price, $gte: Number(minPrice) }; }
    if (maxPrice) { filter.price = { ...filter.price, $lte: Number(maxPrice) }; }
    if (yearFrom || yearTo) {
      filter.year = {};
      if (yearFrom) filter.year.$gte = Number(yearFrom);
      if (yearTo) filter.year.$lte = Number(yearTo);
    }
    const records = await Record.find(filter);
  } catch (error) {
    //...
  }
});
```

Figura 8: gestione del catalogo e dei filtri

Le rotte di consultazione supportano diversi **parametri di query opzionali**, che permettono di filtrare i risultati senza dover definire endpoint separati. In particolare, è possibile effettuare:

- Una ricerca testuale su titolo o artista
- Un filtro per genere musicale
- Un filtro per fascia di prezzo (minimo o massimo)
- Un filtro per intervallo di anni di pubblicazione

Dal punto di vista implementativo, il backend costruisce dinamicamente un oggetto “filter” in base ai parametri nella query string e utilizza tale oggetto in una query “*Record.find(filter)*” per recuperare dal database esclusivamente i prodotti che soddisfano i criteri richiesti. Questo approccio rende l’API flessibile e facilmente estendibile, permettendo al frontend di combinare più criteri di ricerca in un’unica richiesta.

Il filtro per **genere** è trattato separatamente rispetto agli altri filtri, in quanto basato su un insieme finito di valori dipendenti dal contenuto del database a differenza dei filtri per anno o prezzo che operano su intervalli numerici.

Per questo motivo il backend espone una rotta dedicata per il recupero dei generi disponibili, utilizzata dal frontend per costruire dinamicamente l'interfaccia di navigazione e mantenere la coerenza con i dati reali presenti nel catalogo.

Gestione e visualizzazione dei dati su MongoDB

Per facilitare la visualizzazione e la gestione dei dati persistenti dell'applicazione è stato utilizzato **MongoDB Compass**, un'interfaccia grafica che consente di interagire direttamente con i database MongoDB senza ricorrere esclusivamente a query da terminale o a operazioni tramite API.

MongoDB Compass viene connesso alla stessa istanza MongoDB utilizzata dal backend dell'applicazione, tramite la medesima stringa di connessione (MONGO_URI). In questo modo è possibile visualizzare in tempo reale i dati effettivamente utilizzati dal server Express durante l'esecuzione del sito. Una volta stabilita la connessione, Compass mostra l'elenco dei database disponibili e, all'interno di quello relativo al progetto, le varie collection utilizzate dal sistema.

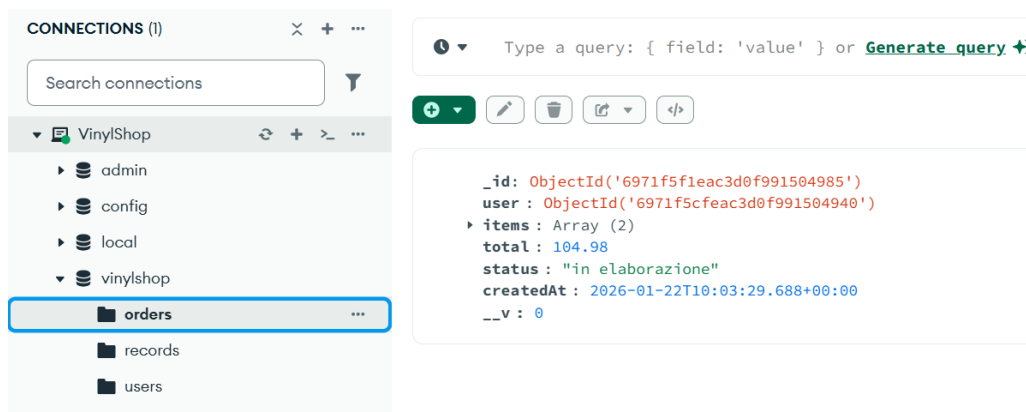


Figura 9: interfaccia MongoDB Compass per la gestione dei dati

Grazie all'interfaccia grafica di MongoDB Compass è possibile non solo visualizzare i dati memorizzati nel database, ma anche **modificare e aggiungere manualmente nuovi elementi** all'interno delle varie collection. In particolare, durante la fase di sviluppo è stato possibile inserire nuovi vinili, aggiornare campi esistenti o verificare in tempo reale le modifiche apportate agli utenti e agli ordini. Queste operazioni sono risultate utili per il popolamento iniziale del database e per il testing dell'applicazione, pur mantenendo coerenza con la struttura dei dati utilizzata dal backend.

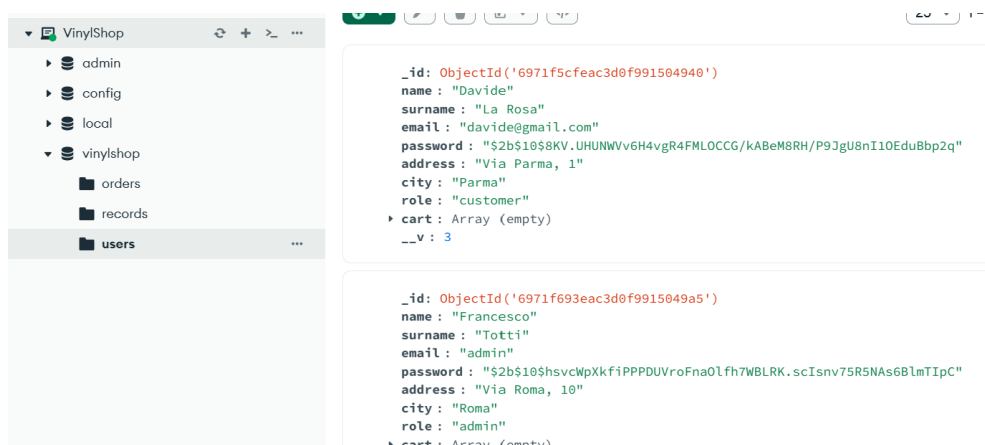


Figura 10: utenti presenti nel DB (password hashata)

È importante sottolineare che MongoDB Compass rappresenta uno strumento di supporto allo sviluppo e non sostituisce le API REST del backend, che rimangono l'unico punto di accesso previsto per il frontend durante il normale utilizzo dell'applicazione.

Thunder Client e test API

Nelle fasi iniziali di sviluppo del backend, prima dell'integrazione con il frontend, il corretto funzionamento delle API REST è stato verificato utilizzando **Thunder Client**, un'estensione di Visual Studio Code che consente di inviare richieste http e analizzarne le risposte in modo semplice immediato.

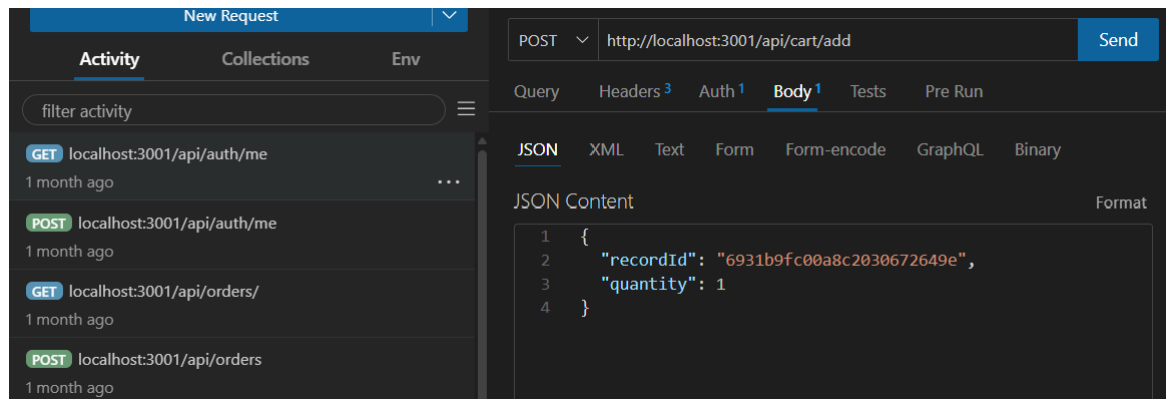


Figura 11: esempio di richiesta http con Thunder Client

Attraverso Thunder Client è stato possibile testare singolarmente gli endpoint del backend, verificando la corretta gestione dei metodi http (GET, POST, PUT, DELETE), il formato delle richieste e delle risposte in JSON e il comportamento del server in presenza di errori. In particolare, sono state testate le API di autenticazione, gestione dei vinili, carrello e ordini, simulando sia richieste pubbliche sia richieste protette da token JWT.

L'utilizzo di questo strumento ha permesso di validare la logica del backend in modo indipendente dal frontend, facilitando il debug e garantendo che le API fossero correttamente funzionali e coerenti con i requisiti prima della loro integrazione nell'interfaccia utente.

AVVIO DEL SERVER

Per poter utilizzare correttamente il sito è necessario avviare separatamente backend e frontend, poiché l'applicazione segue un'architettura Client-Server: il backend espone delle **API REST**, utilizzate dal frontend per recuperare dati, autenticare l'utente e gestire il carrello e gli ordini. Prima dell'avvio del progetto è necessario disporre dei seguenti requisiti:

- **Node.js** (runtime JavaScript) e **npm** (**N**ode **P**ackage **M**anager, gestore dei pacchetti)
- Un'istanza di **MongoDB** (in locale oppure tramite servizio cloud come MongoDB Atlas) per la persistenza dei dati

Il backend richiede alcune variabili d'ambiente per connettersi al database e gestire l'autenticazione tramite JWT. In particolare:

- **MONGO_URI**: stringa di connessione al database MongoDB (configurata in *config/db.js*)
- **JWT_SECRET**: chiave segreta utilizzata per firmare e verificare i token JWT (utilizzata nel *middleware/authmiddleware.js*)

Questa chiave è usata anche nel middleware di autenticazione per validare il token ricevuto dal frontend all'interno dell'header http delle richieste protette.

Per avviare il server backend serve:

1. Aprire un terminale nella cartella del backend
2. Installare le dipendenze con ***npm install***
3. Avviare il server con ***npm start***

```
C:\Users\ladav>cd ProgettoTI/backend
C:\Users\ladav\ProgettoTI\backend>npm start

> backend@1.0.0 start
> node src/server.js

[dotenv@17.2.3] injecting env (3) from .env
Server backend in ascolto sulla porta 3001
MongoDB Connected: 127.0.0.1
```

Una volta avviato, il backend mette a disposizione tutte le rotte API necessarie al funzionamento dell'applicazione.

Per avviare il frontend è necessario:

1. Aprire un secondo terminale nella cartella del frontend
2. Installare le dipendenze con ***npm install***
3. Avviare l'applicazione con ***npm run dev***

```
C:\Users\ladav>cd ProgettoTI/frontend/vite-project
C:\Users\ladav\ProgettoTI\frontend\vite-project>npm run dev

> vite-project@0.0.0 dev
> vite

VITE v7.2.7 ready in 672 ms
  ➔ Local:   http://localhost:5173/
  ➔ Network: use --host to expose
  ➔ press h + enter to show help
```

Il frontend è quindi raggiungibile tramite browser all'indirizzo locale mostrato dal terminale e comunica con il backend tramite chiamate http alle API REST.

FRONTEND

Ruolo del frontend nell'applicazione

Il frontend dell'applicazione è realizzato come **Single Page Application (SPA)**, ovvero un'applicazione che aggiorna dinamicamente contenuti e viste senza ricaricare l'intera pagina dal server a ogni navigazione. Questo approccio consente di offrire un'esperienza utente più fluida e reattiva, tipica delle moderne applicazioni web.

L'interfaccia è sviluppata utilizzando **React** ed è organizzata secondo un'architettura a componenti. Ogni pagina principale del sito corrisponde a un componente di tipo "page" (ad esempio *Home*, *Cart* e *RecordDetail*), mentre elementi comuni come la *Navbar* e il *Footer* sono implementati come componenti riutilizzabili e condivisi tra più pagine.

La comunicazione con il backend avviene tramite richieste http verso le API REST esposte dal server. I dati ricevuti in risposta vengono utilizzati per aggiornare dinamicamente lo stato dei componenti e, di conseguenza, l'interfaccia utente.

Routing e struttura delle pagine

La navigazione tra le pagine è gestita tramite un **sistema di routing** basato React (*src/App.jsx*), che associa un percorso URL a uno specifico componente. Questo permette di definire in modo chiaro e modulare le informazioni di un singolo prodotto, mentre le sezioni carrello, ordini e account gestiscono le funzionalità di acquisto e i dati personali dell'utente.

```
function App() {
  return (
    //così App decide quale componente mostrare in base alla rotta
    <Routes>

      <Route path="/" element={<Layout />} >
        <Route index element={<Home />} />
        <Route path="records/:id" element={<RecordDetail />} />
        <Route path="auth" element={<Auth />} />
        <Route path="about" element={<About />} />

      { /* rotte protette */ }
      <Route element={<ProtectedRoute />} >
        <Route path="cart" element={<Cart />} />
        <Route path="order" element={<Order />} />
        <Route path="account" element={<Account />} />
        <Route path="myorders" element={<MyOrders />} />
      { /* rotte admin */ }
      <Route element={<AdminRoute />} >
        <Route path="admin/records" element={<AdminRecords />} />
        <Route path="admin/records/new" element={<AdminRecordForm mode="create"/>} />
        <Route path="admin/records/edit/:id" element={<AdminRecordForm mode="edit"/>} />
      </Route>
    </Routes>
  );
}
```

Figura 12: gestione delle rotte tramite Routing URL

Layout condiviso: Navbar e Footer

Per mantenere coerenza visiva e ridurre duplicazioni di codice, l'applicazione utilizza un **layout condiviso**. I componenti *Navbar.jsx* e *Footer.jsx* vengono montati una sola volta e rimangono visibili durante la navigazione, mentre il contenuto centrale della pagina varia in base alla rotta selezionata.

Questo approccio migliora l'esperienza utente, garantendo continuità nella navigazione, e semplifica la manutenzione del codice, poiché eventuali modifiche agli elementi comuni possono essere effettuate in un unico punto.

```

export default function Layout() {
  const location = useLocation();

  //rotte senza navbar e footer
  const hideNavbarFooter = ["/login", "/register", "/order", "/auth"];

  //rotte senza footer
  const hideFooterOnly = ["/cart"];

  const hideNavbar = hideNavbarFooter.includes(location.pathname);
  const hideFooter = hideNavbarFooter.includes(location.pathname) || hideFooterOnly.includes(location.pathname);

  return (
    <>
    {!hideNavbar && <Navbar />}
    <main style={{ padding: "18px 16px" }}>
    <Outlet />
    </main>
    {!hideFooter && <Footer />}
    </>
  );
}

```

Figura 13: configurazione della Navbar e del Footer

```

{user?.role === "admin" && (
  <Link to="/admin/records">Gestione vinilli</Link>
)}

<div className="navTopRight">
  {token ? (
    <>
    <div className="navAccountWrap">
      <Link to="/account" className="navAccount">
        <FaUserCircle />
        <span>Account</span>
      </Link>
    </div>
    </>
  ) : (
    <Link to="/auth" className="navAccount">
      <FaUserCircle />
      Sign in
    </Link>
  )}
  <Link to="/cart"
    className="navIconBtn"
    aria-label="Carrello"
  >

```

Figura 14: esempio di link associati alla Navbar

Comunicazione con il backend

Per evitare di ripetere in ogni pagina la stessa logica di comunicazione (base URL, gestione degli headers e degli errori), le chiamate http sono **centralizzate** in un modulo dedicato (*api/api.js*). Questo modulo funge *da strato di accesso alle API REST del server*, incapsulando la logica delle richieste http e gestendo automaticamente l'inserimento degli header necessari, l'eventuale token JWT per le rotte protette e le risposte in formato JSON.

```

export async function apiFetch(
  path,
  { method = "GET", body = undefined } = {}
) {
  const headers = {
    "Content-Type": "application/json", //header standard per le richieste JSON
  };

  //token sempre incluso automaticamente (se esiste viene sempre inviato)
  const token = getToken();
  if (token) {
    headers["Authorization"] = `Bearer ${token}`;
  }

  //richiesta http al backend
  const response = await fetch(`${API_URL}${path}`, {
    method,
    headers,
    //se body esiste, converto in JSON
    body: body
      ? (typeof body === "string" ? body : JSON.stringify(body))
      : undefined,
  });
}

```

Figura 15: apiFetch è la funzione che permette la centralizzazione

I componenti React non interagiscono direttamente con gli endpoint del backend, ma utilizzano le funzioni offerte da *api.js*, migliorando la leggibilità del codice e la separazione delle responsabilità, permettendo ai componenti di concentrarsi sulla logica di prestazione e demandando la comunicazione con il server al modulo centralizzato (ad esempio tramite funzioni come “getRecords()”, “getRecordById()” o “addToCart()”).

```
try {
  await apiFetch("/cart/add", {
    method: "POST",
    body: { recordId: id, quantity: qty }
  });
  window.dispatchEvent(new Event("cart-updated"));
  setMessage("Vinile aggiunto al carrello");
}
```

Figura 16: esempio di chiamata POST per aggiungere un elemento al carrello

Autenticazione nel frontend

La gestione dell'autenticazione comprende pagina dedicata alla **registrazione** e al **login** (*/auth*). In seguito a un login valido, il token JWT restituito dal backend viene salvato nel **localStorage** del browser.

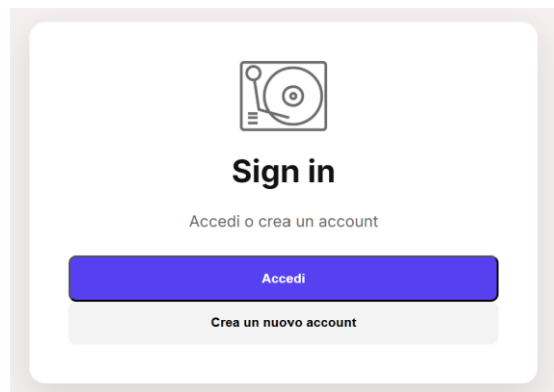


Figura 17: struttura di accesso

Questo token viene successivamente riutilizzato dal modulo *api.js* per autenticare le richieste verso le API protette, come quelle relative al carrello (*/cart*) e agli ordini (*/orders*). In questo modo, il frontend mantiene lo stato di autenticazione dell'utente anche durante la navigazione tra le diverse pagine dell'applicazione.

```
/* LOGIN */
const handleLogin = async () => {
  setError("");
  try {
    const res = await apiFetch("/auth/login", {
      method: "POST",
      body: { email, password }
    });

    localStorage.setItem("token", res.token);
    navigate("/");
  } catch (err) {
    setError(err.message);
  }
};
```

Figura 18: gestione del login con chiamata ad *api.js* per l'autenticazione

Protezione delle pagine

Alcune sezioni dell'applicazione, come il carrello, la visualizzazione degli ordini e l'area account, sono accessibili solo a utenti autenticati. Per gestire questo comportamento è stato introdotto un componente di protezione (*/components/ProtectedRoute.jsx*), che verifica la presenza del token di autenticazione (o dello stato utente) prima di consentire l'accesso alla pagina richiesta.

```
/* rotte protette */
<Route element={<ProtectedRoute />} >
  <Route path="cart" element={<Cart />} />
  <Route path="order" element={<Order />} />
  <Route path="account" element={<Account />} />
  <Route path="myorders" element={<MyOrders />} />
</Route>
```

Figura 19: gestione delle rotte protette in App.jsx

In caso di assenza di autenticazione, l'utente viene automaticamente reindirizzato alla pagina di login. Questo meccanismo impedisce l'accesso alle funzionalità riservate dal punto di vista dell'interfaccia utente, migliorando l'esperienza di navigazione e mantenendo coerenza con i controlli di sicurezza implementati nel backend.

```
export default function ProtectedRoute() {
  const token = localStorage.getItem("token");
  const location = useLocation();

  if(!token) {
    //reindirizza a /auth
    return <Navigate to="/auth" replace state={{ from: location.pathname }} />;
  }

  return <Outlet />; //rendere le rotte "figlie"
}
```

Figura 20: controllo del token

Home: catalogo, ricerca e filtri combinati

La pagina **Home** (*/Home.jsx*) rappresenta il punto di ingresso principale dell'applicazione e mostra il catalogo completo dei vinili disponibili. Dal punto di vista funzionale, il frontend mantiene lo stato dei filtri selezionati dall'utente, tra cui genere musicale, intervallo di prezzo, intervallo di anni e testo di ricerca.

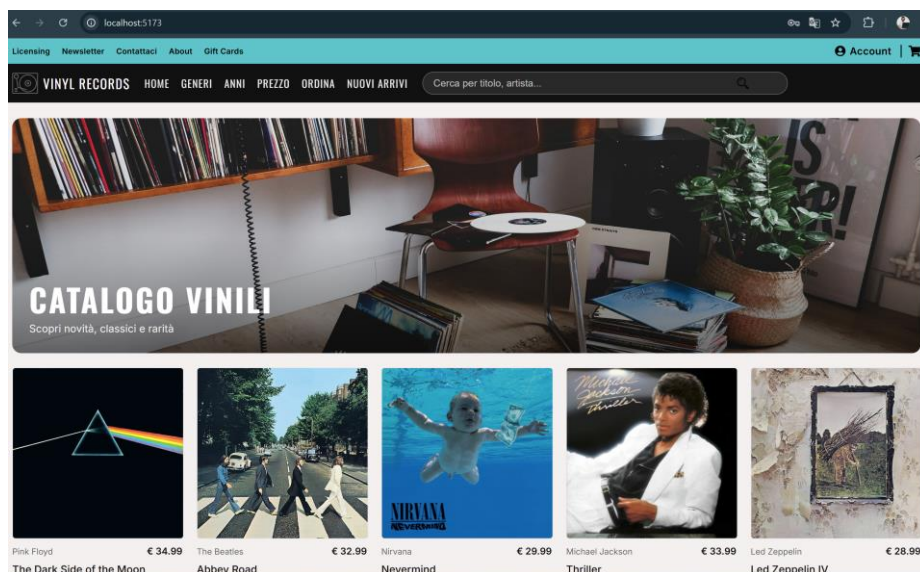


Figura 21: Home del sito

Sulla base di questi parametri, il frontend costruisce dinamicamente una query e invia una richiesta al backend tramite l'endpoint GET `/api/records`, utilizzando la query string per specificare i filtri attivi.

Ogni volta che uno dei filtri viene modificato, la richiesta viene ripetuta e la lista dei prodotti visualizzati viene aggiornata di conseguenza, garantendo che il catalogo mostrato sia sempre coerente con la combinazione di criteri selezionati.

Lista generi: utilizzo dell'endpoint “meta/genres”

La selezione di un genere, così come per gli altri filtri, non comporta il caricamento di una nuova pagina nel senso tradizionale, ma aggiorna lo stato dei filtri attivi nella Home. Di conseguenza, viene inviata una nuova richiesta al backend per ottenere esclusivamente i vinili appartenenti al genere selezionato, mantenendo il comportamento tipico di una Single Page Application.

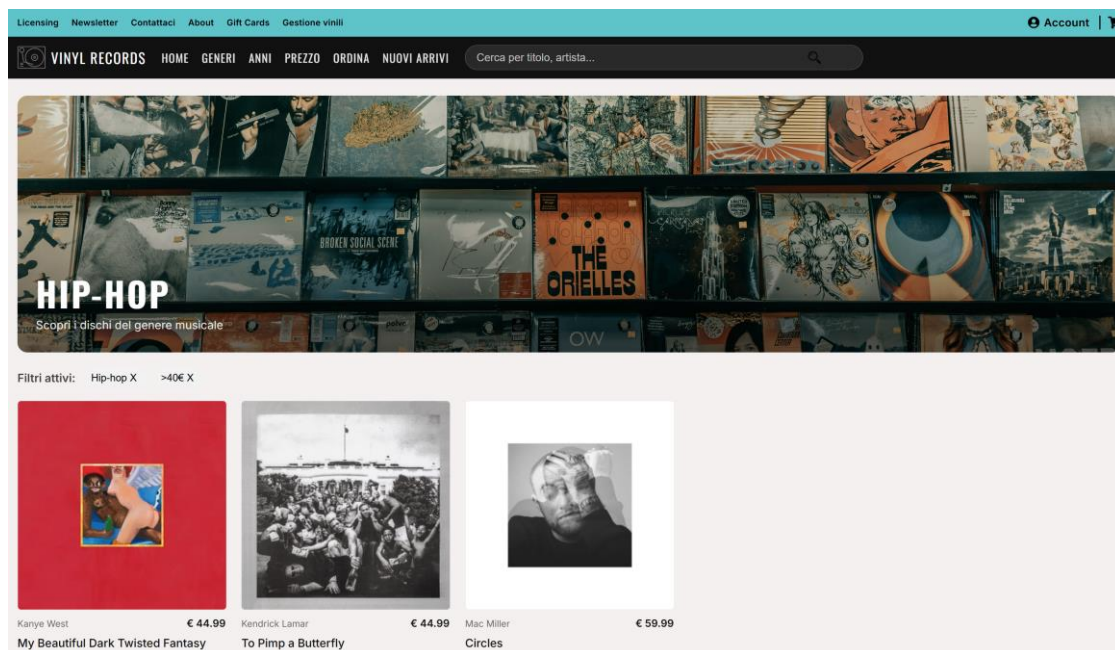


Figura 22: applicazione di filtri per genere e prezzo

Sappiamo che nel backend il filtro per **genere** è gestito tramite un endpoint separato (*meta/genres*) perché, a differenza degli altri criteri di filtraggio, si basa su un **insieme finito e discreto di valori** che dipendono direttamente dai dati presenti nel database.

Mentre filtri come prezzo e anno operano su intervalli numerici e possono essere gestiti direttamente tramite parametri di query (`minPrice`, `maxPrice`, `yearFrom`, `yearTo`), il genere rappresenta una categoria il cui elenco non è noto a priori dal frontend.

RecordDetail: dettaglio vinile e aggiunta al carrello

La pagina di dettaglio del vinile (*/records/:id*) consente di visualizzare le informazioni complete di un singolo prodotto, recuperato dal backend tramite l'identificativo presente nell'URL. Una volta ottenuti i dati, l'interfaccia mostra la copertina, le informazioni principali, una breve descrizione, la tracklist e il pulsante per l'aggiunta al carrello.

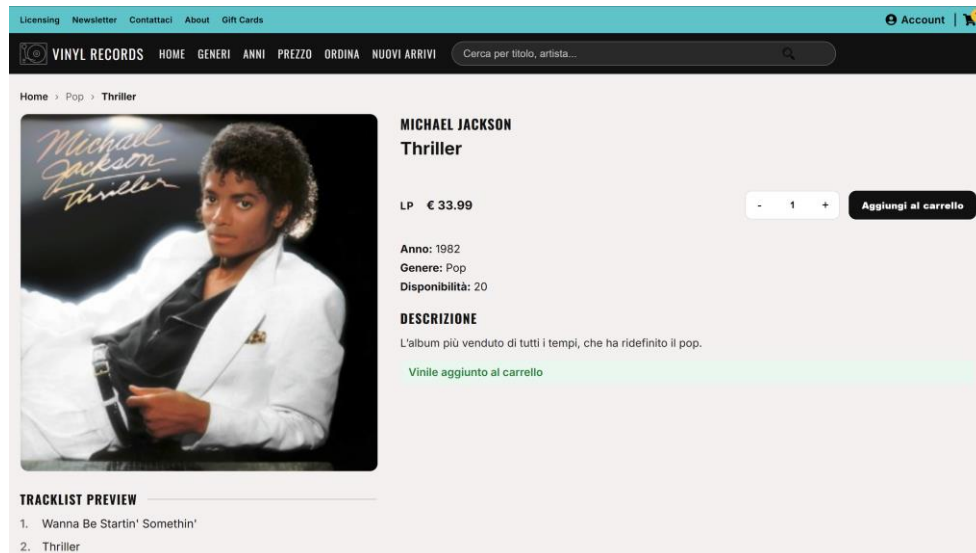


Figura 23: pagine di dettaglio del prodotto

L'azione di aggiunta al carrello genera una richiesta verso il backend per aggiornare il carrello dell'utente autenticato. A seguito di una risposta positiva, l'interfaccia fornisce un feedback all'utente tramite un messaggio di conferma; altrimenti reindirizza l'utente alla pagina di login/registrazione per poter proseguire con l'operazione.

All'interno della pagina di dettaglio (*RecordDetail.jsx*) è presente una sezione dedicata ai prodotti consigliati, composta da altri vinili appartenenti allo stesso genere del prodotto attualmente visualizzato. Questa funzionalità migliora l'esperienza di navigazione, suggerendo all'utente prodotti affini e incentivando l'esplorazione del catalogo.

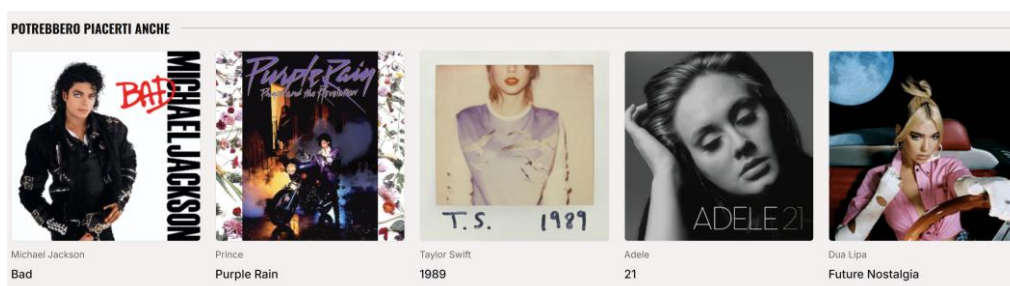


Figura 24: esempio di vinili consigliati in base al genere Pop

Dal punto di vista implementativo, il frontend invia al backend una richiesta utilizzando il genere del vinile corrente come parametro di filtro e riceve una lista di record con lo stesso valore di *genre*. I prodotti restituiti vengono quindi renderizzati nella sezione dedicata, posizionata al di sotto del dettaglio principale.

Carrello: visualizzazione, modifica quantità e rimozione

La pagina del carrello (*Cart.jsx*) consente di visualizzare il contenuto del carrello dell'utente autenticato, recuperato dal backend tramite API dedicate. I prodotti vengono mostrati in forma di lista, con indicazioni della quantità selezionata e del totale complessivo.

Le operazioni di modifica della quantità o di rimozione di un prodotto generano richieste al backend, che aggiorna i dati persistenti associati all'utente. A seguito di ogni operazione, il frontend sincronizza lo stato locale con la risposta ricevuta, garantendo coerenza tra l'interfaccia utente e i dati memorizzati nel database.

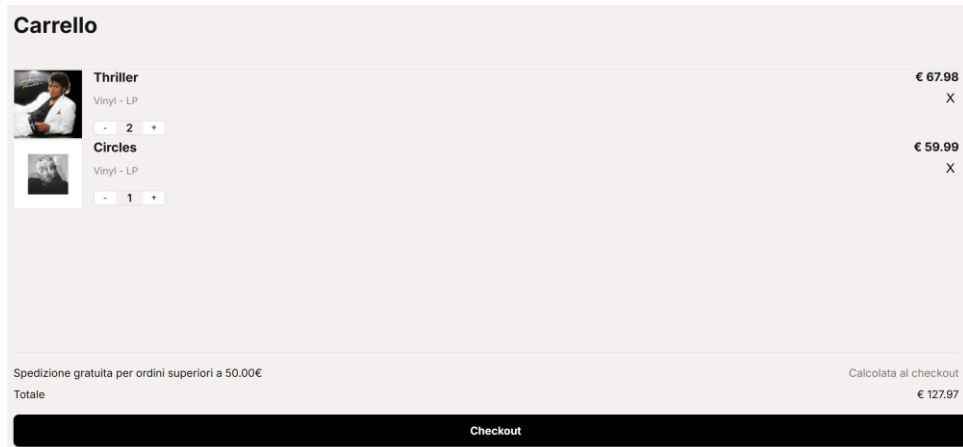


Figura 25: esempio di carrello

Checkout e creazione ordine

La pagina di checkout consente all'utente di confermare i dati relativi all'acquisto e di inviare al backend la richiesta di creazione di un nuovo ordine. In questa fase, il backend genera un documento *Order.jsx* contenente i prodotti acquistati e svuota il carrello dell'utente. Viene inoltre applicato un cost di spedizione aggiuntivo sul totale, gratuito in caso il prezzo degli articoli superi i 50€ di importo.

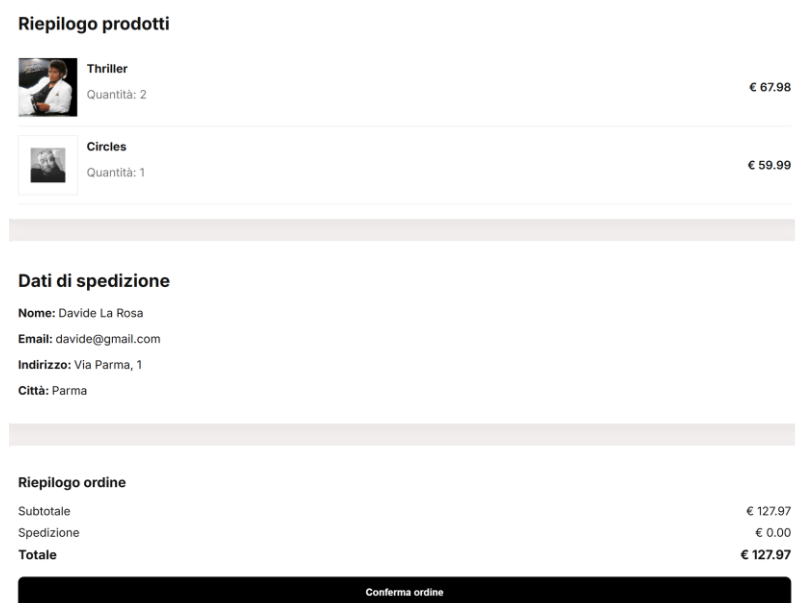


Figura 26: esempio di pagine checkout con la conferma

Dopo una risposta positiva, il frontend mostra una conferma dell'operazione, aggiorna lo stato locale (carrello vuoto e ordine creato) e reindirizza l'utente alla pagina dedicata alla visualizzazione degli ordini effettuati.

Account e MyOrders: dati utente e ordini passati

La pagina Account (*Account.jsx*) raccoglie le informazioni principali dell'utente autenticato e fornisce accesso alle sezioni personali, come il profilo, lo storico degli ordini e il logout. All'interno di questa sezione è inoltre possibile modificare alcuni dati personali, come le informazioni di indirizzo.

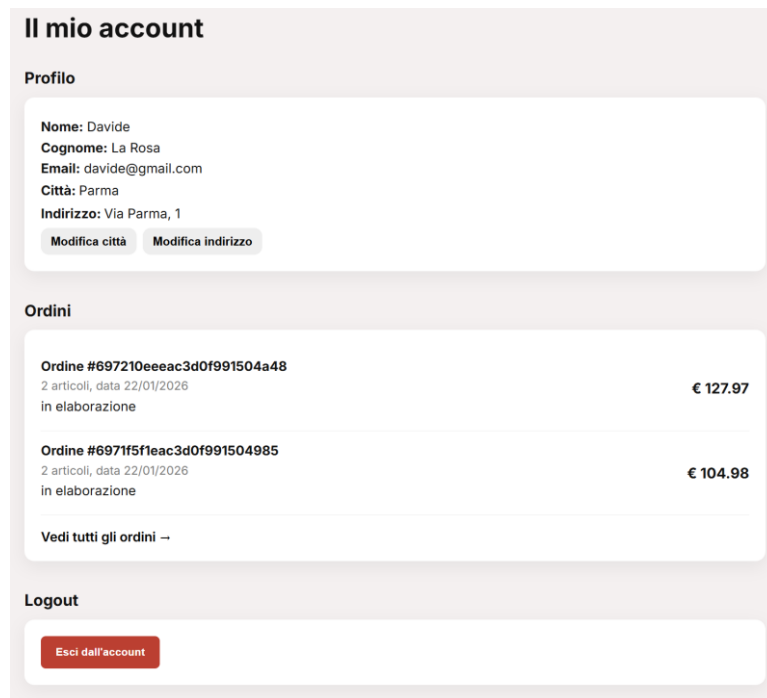


Figura 27: sezione MyAccount per vedere le informazioni dell'utente

La pagina MyOrders mostra l'elenco degli ordini effettuati dall'utente, recuperati dal backend e renderizzati come lista. Per ciascun ordine vengono visualizzati i dettagli principali, come la data di creazione, l'importo totale e i prodotti acquistati. Questa sezione completa il **ciclo di acquisto**, permettendo all'utente di consultare in qualsiasi momento lo storico e lo stato delle operazioni effettuate.

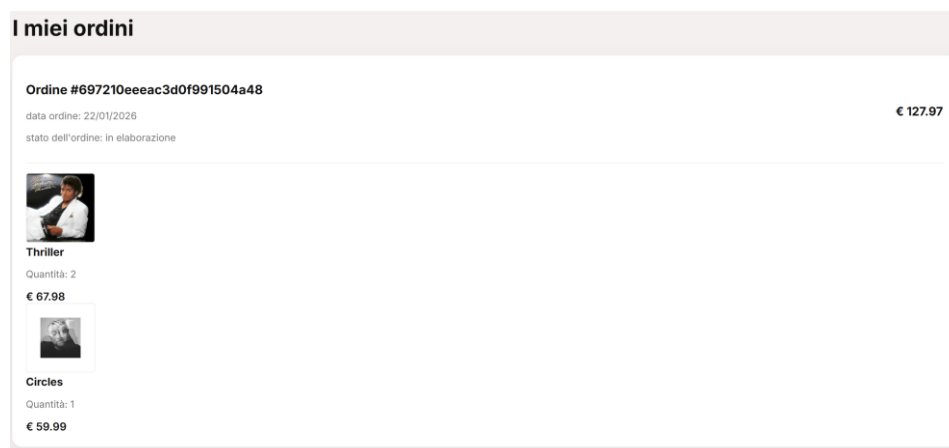


Figura 28: visualizzazione dettagliata dell'ordine

Sezione amministratore e gestione del catalogo

Il frontend include una sezione (*/admin/records*) dedicata agli **utenti con ruolo di amministratore**, che consente la gestione completa del catalogo dei vinili presenti nello shop. Questa sezione fornisce un'interfaccia grafica semplice e intuitiva per le operazioni amministrative, permettendo di visualizzare, creare ed eliminare i record presenti nel database.

Tali operazioni corrispondono alle **funzionalità CRUD** (Create, Read, Update, Delete) esposte dal backend e protette da meccanismi di autorizzazione.

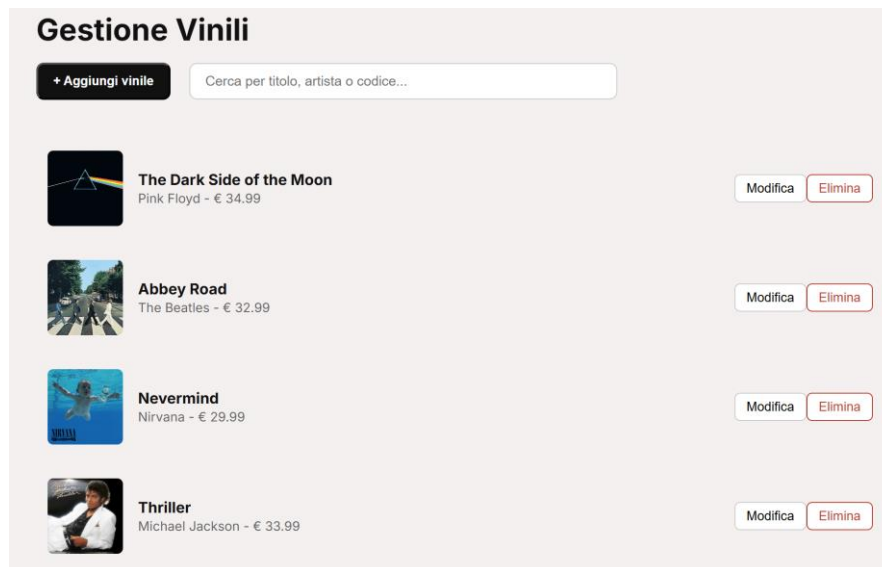


Figura 29: sezione dedicata alla gestione del catalogo per l'admin

Dal punto di vista funzionale, la sezione amministratore rappresenta quindi il frontend delle **API CRUD sui record**, consentendo la gestione del catalogo senza la necessità di intervenire direttamente sul database.

```
//componente per proteggere le rotte admin
export default function AdminRoute() {
  const [loading, setLoading] = useState(true);
  const [isAdmin, setIsAdmin] = useState(false);

  useEffect(() => {
    const checkAdmin = async () => {
      try {
        const user = await apiFetch("/auth/me");
        setIsAdmin(user.role === "admin");
      } catch {
        setIsAdmin(false);
      } finally {
        setLoading(false);
      }
    };

    checkAdmin();
  }, []);
}
```

Figura 30: verifica del ruolo di admin per l'accesso alle funzionalità amministrative

Accesso e protezione delle funzionalità amministrative

Le pagine amministrative sono accessibili solo agli utenti autenticati con ruolo **admin** (*AdminRecord.jsx*). Il frontend verifica lo stato di autenticazione e il ruolo dell'utente prima di consentire l'accesso alle rotte riservate.

```
{/* rotte admin */}
<Route element={<AdminRoute />} >
  <Route path="admin/records" element={<AdminRecords />} />
  <Route path="admin/records/new" element={<AdminRecordForm mode="create"/>} />
  <Route path="admin/records/edit/:id" element={<AdminRecordForm mode="edit"/>} />
</Route>
```

Figura 31: rotte admin gestite in App.jsx

È importante sottolineare che questo controllo nel frontend ha lo scopo di migliorare l'esperienza utente, mentre la protezione effettiva delle risorse è sempre garantita dal backend tramite middleware di autorizzazione.

Creazione e modifica dei vinili

La creazione e la modifica dei vinili sono gestite tramite un **form riutilizzabile** (*AdminRecordForm.jsx*), che opera in due modalità distinte:

- **Create**: per l'inserimento di un nuovo vinile
- **Edit**: per la modifica di un vinili esistente

In modalità modifica, il form recupera automaticamente i dati del vinile selezionato tramite una chiamata al backend sull'identificativo presente nell'URL e precompila i campi, consentendo all'amministratore di aggiornare solo le informazioni necessarie.

Per la gestione della tracklist, il frontend permette l'inserimento delle tracce come stringa separata da virgole, che viene successivamente convertita in un array prima dell'invio al backend, mantenendo la coerenza con il modello dei dati.

The image shows a web form titled "Modifica Vinile" (Modify Vinyl). It contains several input fields, each with a label and a value. The fields are: "Titolo:" with "Nevermind", "Artista:" with "Nirvana", "Anno:" with "1991", "Genere:" with "Grunge", "Prezzo:" with "29,99", "Stock:" with "12", "URL Immagine:" with "https://upload.wikimedia.org/wikipedia/en/b/b7/NirvanaNevermindalbumcover.jpg", "Descrizione:" with "Album simbolo del grunge che ha portato il rock alternativo nel mainstream.", and "Tracce (separate da virgola):" with "Smells Like Teen Spirit, In Bloom, Come as You Are, Breed, Lithium, Polly, Drain You, Som". At the bottom right, there are two buttons: "Annulla" (Cancel) and "Salva Vinile" (Save Vinyl).

Figura 32: esempio di modifica di un vinile

Gestione dell'invio e validazione dei dati

Alla conferma del form, i dati vengono prima normalizzati (ad esempio conversione di prezzo e anno in valori numerici) e successivamente inviati al backend tramite una richiesta http (POST o PUT a seconda della modalità). In caso di esito positivo, l'utente viene reindirizzato alla lista dei vinili. Il frontend fornisce inoltre un riscontro positivo durante il salvataggio, mostrando lo stato di caricamento e segnalando eventuali errori, migliorando così l'usabilità complessiva della sezione amministrativa.

Eliminazione dei vinili

L'eliminazione di un vinile avviene direttamente dalla lista amministrativa ed è preceduta da una richiesta di conferma, al fine di evitare cancellazioni accidentali. Dopo la conferma, il backend rimuove il documento dal database e il frontend aggiorna la visualizzazione della lista dei vinili.

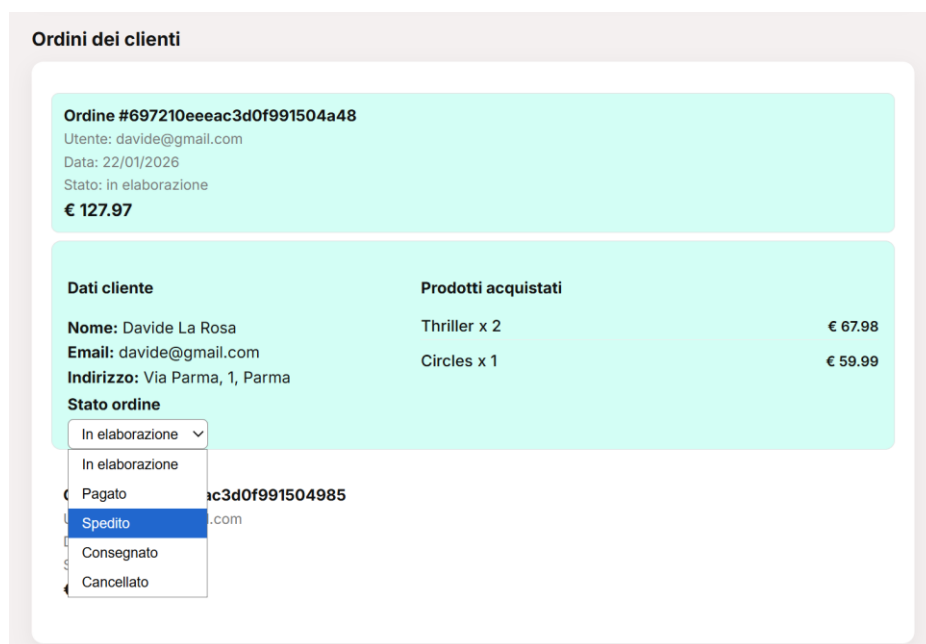
Questa operazione completa il **ciclo CRUD** e dimostra l'integrazione completa tra frontend amministrativo e backend protettivo.

Gestione ordini lato amministrativo

Oltre alla gestione del catalogo, l'area amministrativa consente anche la **gestione degli ordini effettuati** dagli utenti. In particolare, un utente con ruolo admin può visualizzare l'elenco completo degli ordini presenti nel sistema, indipendentemente dall'utente che li ha generati.

Dal punto di vista funzionale, questa sezione permette all'amministratore di:

- Consultare i dettagli di ciascun ordine (utente, data, prodotti acquistati e totale)
- Monitorare ed aggiornare lo stato dell'ordine, simulando il flusso reale di gestione post-acquisto



Ordini dei clienti

Ordine #697210eeeac3d0f991504a48
Utente: davide@gmail.com
Data: 22/01/2026
Stato: In elaborazione
€ 127.97

Dati cliente	Prodotti acquistati
Nome: Davide La Rosa	Thriller x 2 € 67.98
Email: davide@gmail.com	Circles x 1 € 59.99
Indirizzo: Via Parma, 1, Parma	

Stato ordine
In elaborazione ▼

- In elaborazione
- Pagato
- Spedito**
- Consegnato
- Cancellato

Figura 33: gestione degli ordini da parte dell'admin

Le operazioni amministrative sugli ordini sono protette tramite autenticazione e autorizzazione: l'accesso alle API dedicate richiede un token JWT valido e il controllo del ruolo admin. L'aggiornamento dello stato avviene tramite una richiesta http di modifica verso il backend; in caso di esito positivo, il frontend aggiorna la visualizzazione mostrando lo stato corrente dell'ordine.

Questa funzionalità completa la gestione amministrativa del sito, distinguendo chiaramente tra le operazioni disponibili all'utente finale e quelle riservate all'amministratore.

Stile e gestione del layout (CSS)

Per la realizzazione dell'interfaccia grafica dell'applicazione è stato utilizzato **CSS (Cascading Style Sheets)** per la definizione dello stile, del layout e dell'aspetto visivo delle diverse pagine. Lo stile è stato progettato con l'obiettivo di garantire coerenza grafica, leggibilità e usabilità, mantenendo un aspetto semplice e uniforme in tutte le sezioni del sito.

Il CSS è organizzato in modo modulare, con fogli di stile associati alle principali pagine o componenti dell'applicazione. Questo approccio consente di separare la logica di presentazione dalla logica applicativa e di mantenere il codice ordinato e manutenibile.

Le regole di stile sono state utilizzate per definire una struttura coerente tra le diverse pagine, mantenendo elementi comuni come tipografia, colori e spaziature. In particolare, il layout è stato pensato per valorizzare il catalogo dei vinili e facilitare la navigazione dell'utente, con una chiara distinzione tra contenuti principali e sezioni secondarie.

Griglie e contenitori flessibili sono stati impiegati per organizzare le liste di prodotti e le sezioni di dettaglio, migliorando la leggibilità e la disposizione degli elementi all'interno della pagina.

Ruolo del CSS nel progetto

Il CSS non ha avuto un ruolo puramente estetico, ma è stato utilizzato come strumento fondamentale per migliorare l'**esperienza utente complessiva**, supportando la chiarezza dell'interfaccia e rendendo le interazioni più intuitive. La progettazione dello stile è stata affrontata tenendo conto non solo delle funzionalità dell'applicazione, ma anche di principi di usabilità e leggibilità.

Per la realizzazione di questa parte del progetto è stata svolta anche una fase di **approfondimento esterno e personale**, raccogliendo informazioni e consigli su buone pratiche di *design* e *user experience* tramite persone che studiano l'effettiva esperienza utente dei siti web. Questo confronto ha permesso di prendere decisioni più consapevoli nella disposizione degli elementi, nella gestione degli spazi, nella scelta delle dimensioni e nella gerarchia visiva dei contenuti, cercando di rendere l'interfaccia semplice, ma il più possibile intuitiva e piacevole da utilizzare.

La separazione tra struttura (HTML/JSX), logica applicativa (JavaScript/React) e presentazione (CSS) ha inoltre permesso di mantenere il progetto ordinato, facilmente manutenibile ed estendibile a nuove funzionalità.

STRUTTURA DEL PROGETTO

Organizzazione del codice, gestione delle risorse e considerazioni finali

Nel corso dello sviluppo del progetto è stata posta particolare attenzione non solo alla realizzazione delle funzionalità richieste, ma anche all'organizzazione del codice e alle scelte progettuali che ne hanno guidato la struttura complessiva. L'obiettivo è stato quello di mantenere il sistema comprensibile, modulare e coerente, facilitando sia lo sviluppo sia la manutenzione futura.

Nel frontend dell'applicazione sono state utilizzate diverse funzioni e costrutti messi a disposizione da React e dall'ecosistema JavaScript, con l'obiettivo di gestire in modo efficiente lo stato dell'interfaccia, le operazioni asincrone e la comunicazione con il backend.

Le funzioni di tipo *useState* (con i relativi metodi di aggiornamento *setState*) vengono utilizzate per **gestire lo stato locale dei componenti**, come ad esempio i dati caricati dal backend, i filtri selezionati, il contenuto del carrello o i valori inseriti nel form. Questo approccio consente di aggiornare dinamicamente l'interfaccia ogni volta che lo stato cambia, senza ricaricare la pagina.

Le funzioni *useEffect* sono invece impiegate per **gestire operazioni asincrone ed effetti collaterali**, come il caricamento dei dati iniziali da API REST o l'aggiornamento dei contenuti in risposta a cambiamenti di parametri (ad esempio filtri o identificativi presenti nell'URL). In questo modo il frontend può reagire ai cambiamenti di stato mantenendo sincronizzati interfaccia e dati.

Per la comunicazione con il backend, come visto in precedenza, le chiamate API REST sono **centralizzate in un modulo dedicato** (*api.js*), che espone funzioni specifiche per ciascuna operazione (recupero dei vinili, dettaglio prodotto, gestione del carrello, creazione ordini), tramite la chiamata di *apiFetch*. Questo permette ai componenti React di invocare semplici funzioni JavaScript senza dover gestire direttamente i dettagli delle richieste http, migliorando la leggibilità e la manutenibilità del codice.

Gestione delle immagini

Per quanto riguarda la gestione delle immagini, è stato adottato un approccio differenziato in base alla tipologia delle risorse. Le immagini statiche dell'interfaccia, come icone, sfondi ed elementi grafici comuni, sono caricate localmente all'interno della cartella *public* del frontend, garantendo un caricamento immediato e una maggiore affidabilità delle risorse.

Le immagini di copertina dei vinili, invece, non sono memorizzate localmente ma vengono recuperate tramite URL esterni, in particolare facendo riferimento alle immagini disponibili sulle pagine Wikipedia dei dischi in questione. Questa scelta consente di mantenere il database più leggero e facilitare l'inserimento di nuovi prodotti.

Interfacce particolari: Nuovi arrivi e About

All'interno dell'interfaccia principale è stata introdotta una sezione dedicata ai “nuovi arrivi”, che mostra esclusivamente i vinili pubblicati nell'ultimo anno (2025). Questa funzionalità è stata pensata come un'estensione del catalogo principale, realizzata senza modificare il layout generale dell'applicazione.

Dal punto di vista implementativo, la sezione sfrutta i dati già presenti nel database e applica un filtro sull'anno di pubblicazione, permettendo di valorizzare le uscite più recenti senza introdurre nuove complessità strutturali.

L'applicazione include una sezione *Abuot.jsx*, accessibile dal frontend nella Navbar superiore, che ha lo scopo di fornire una descrizione sintetica del progetto, del suo funzionamento generale e delle tecnologie utilizzate.

Questa pagina è pensata come supporto informativo per l'utente e come riepilogo concettuale dell'applicazione. All'interno della sezione About viene spiegato il flusso principale di utilizzo del sito, il ruolo degli utenti e degli amministratori e l'architettura Client-Server adottata. Ho voluto inoltre presentare una sezione di **disclaimer**, che chiarisce i limiti del sistema: l'applicazione non rappresenta un e-commerce reale, non gestisce transazioni economiche e utilizza contenuti multimediali coperti da copyright esclusivamente a scopo didattico e dimostrativo per dare veridicità al sistema.

Gestione degli errori

Nel progetto è stata posta particolare attenzione alla gestione degli errori e alla comunicazione chiara degli stati di errore all'utente, sia nel frontend che nel backend. I messaggi di errore sono stati pensati sia come **strumento di debug** che come parte integrante dell'esperienza utente, con l'obiettivo di rendere il comportamento dell'applicazione comprensibile anche in situazioni non ideali.

Il backend gestisce le principali condizioni di errore restituendo risposte http coerenti con lo standard REST, accompagnate da messaggi descrittivi in formato JSON. In particolare, nel progetto vengono utilizzati i seguenti codici di stato http:

- **400 (Bad Request)**: utilizzato quando la richiesta del client non è valida, ad esempio in presenza di dati mancanti o formati non corretti durante la creazione o modifica di risorse
- **401 (Unauthorized)**: utilizzato quando un utente tenta di accedere a una risorsa protetta senza fornire un token JWT valido o quando il token è assente o scaduto
- **403 (Forbidden)**: utilizzato nei casi in cui l'utente è autenticato ma non dispone dei privilegi necessari, ad esempio nel tentativo di accedere a funzionalità riservate agli amministratori
- **404 (Not Found)**: restituito quando la risorsa richiesta non esiste, come nel caso di un vinile o di un ordine non presente nel database
- **500 (Internal Server Error)**: utilizzato per gestire errori interni imprevisti del server, come problemi durante l'esecuzione della logica applicativa o errori di connessione nel database

I messaggi associati a questi codici sono chiari e sintetici, evitando di esporre dettagli implementativi interni o informazioni sensibili, in linea con le buone pratiche di sicurezza.

Nel frontend, i codici di errore restituiti dal backend vengono intercettati e gestiti in modo appropriato, traducendoli in **messaggi comprensibili** per l'utente. Questo permette di fornire un feedback immediato in caso di problemi, come ad esempio errori di autenticazione, accesso non autorizzato, risorse non trovate o problemi di comunicazione con il server.

Oltre ai messaggi di errore, il frontend utilizza anche **stati di caricamento** e **messaggi di conferma** per segnalare all'utente quando un'operazione è in corso o quando è stata completata correttamente, riducendo l'ambiguità e migliorando l'usabilità complessiva dell'applicazione.

File di ingresso dell'applicazione

Il file *main.jsx* rappresenta il **punto di ingresso dell'applicazione frontend**. In questo file viene inizializzata l'app React e viene definito il contesto globale di routing tramite *BrowserRouter*, che consente la navigazione tra le diverse pagine senza dover ricaricare il browser. Da qui viene renderizzato il componente principale *App.jsx*, che a sua volta gestisce la struttura generale dell'applicazione e il sistema di rotte.

In *index.css* sono contenute invece le **regole di stile globali dell'applicazione**. In esso vengono definiti gli stili di base comuni a tutte le pagine, come font, colori principali, reset CSS e impostazioni generali del layout. Questo consente di mantenere coerenza visiva tra le diverse sezioni del sito e di separare gli stili globali da quelli specifici dei singoli componenti o pagine.

Limiti del sistema e possibili estensioni future

Il progetto presenta alcuni limiti tecnici, coerenti con il contesto e con gli obiettivi didattici, che è comunque buona prassi evidenziare. In particolare, l'applicazione non gestisce transazioni economiche reali e non include un sistema di pagamento, limitandosi a simulare il flusso di acquisto. La gestione dello stock dei prodotti è anch'essa simulata e non tiene conto di aggiornamenti concorrenti o di una disponibilità in tempo reale.

Dal punto di vista della sicurezza, l'autenticazione è basata su token JWT senza l'implementazione di meccanismi più avanzati, come *refresh token* o sistemi di *rate limiting*. Nel progetto il token JWT viene utilizzato come unico strumento di identificazione dell'utente, mentre i refresh token permetterebbero una gestione più avanzata delle sessioni nel tempo e il rate limiting consentirebbe di limitare il numero di richieste per prevenire abusi; tali meccanismi non sono necessari e non sono stati introdotti, al fine di mantenere una complessità coerente con lo scopo didattico dell'applicazione.

Nonostante questi limiti, il sistema si presta a diverse possibili estensioni future. Tra le funzionalità che potrebbero essere integrate si possono citare: un sistema di valutazione dei vini da parte degli utenti, la gestione di una wishlist personale, l'introduzione di pagamenti simulati e una gestione più accurata dello stock dei prodotti.

Queste e altre funzionalità non sono state implementate per una scelta progettuale. In particolare, si è preferito concentrare il lavoro sulla solidità dell'architettura Client-Server e sulla corretta gestione dei dati, evitando di introdurre elementi che avrebbero aumentato la complessità senza apportare benefici significativi allo scopo didattico del progetto.

CONCLUSIONI

In conclusione, il progetto mi ha permesso di applicare in modo concreto le conoscenze acquisite durante il corso di Tecnologie Internet del professore Michele Amoretti (UNIPR), integrando aspetti di sviluppo frontend, backend, gestione dei dati e progettazione dell'architettura di un'applicazione web completa. L'obiettivo principale è stato quello di realizzare un sistema coerente e funzionante, che simulasse il comportamento di un e-commerce reale, mantenendo però una complessità adeguata allo scopo didattico.

Durante lo sviluppo del progetto ho posto particolare attenzione non solo all'implementazione tecnica, ma anche alla comprensione delle scelte progettuali, alla struttura del codice e alla manutenibilità dell'applicazione. Per questo motivo, il lavoro è stato accompagnato da una fase costante di studio e approfondimento, facendo riferimento ad appunti delle lezioni, materiale didattico del corso, documentazione ufficiale delle tecnologie utilizzate e ricerche mirate su fonti attendibili.

Un ruolo importante è stato svolto anche dal confronto esterno e dall'organizzazione del lavoro: la pianificazione delle funzionalità, la suddivisione in moduli e la progressiva integrazione delle diverse parti del sistema hanno contribuito a mantenere il progetto ordinato e coerente nel tempo.

Nel corso della realizzazione è stato inoltre fatto uso di strumenti di Intelligenza Artificiale come supporto allo sviluppo, in particolare per facilitare la fase di stesura della documentazione, l'organizzazione di idee, la pianificazione delle funzionalità e il chiarimento di concetti tecnici. Tali strumenti sono stati utilizzati come supporto e accelerazione del processo di apprendimento, senza sostituire la comprensione personale del codice e delle soluzioni adottate, che sono sempre state analizzate, adattate e integrate in modo consapevole, creativo e personale all'interno del progetto.

Nel complesso, il progetto rappresenta un'esperienza formativa significativa, che ha permesso di consolidare competenze tecniche e metodologiche, offrendo una visione più completa dello sviluppo di applicazioni web moderne e delle problematiche reali legate alla loro progettazione, all'implementazione e agli aspetti di sicurezza, affrontati anche nell'ambito della cybersecurity prevista dal corso.

Davide La Rosa - 344247